

Konstantin Lucius Regenhardt

Entwurf und Evaluation eines automatisierten Migrationswerkzeugs von Katalon-Groovy-Tests zu Selenium-Pytest

Bachelorarbeit eingereicht im Rahmen der Bachelorprüfung
im Studiengang *Bachelor of Science Umweltinformatik*
am Department Fachbereich 2
der Fakultät Technik und Informatik
der Hochschule für Technik und Wirtschaft Berlin

Betreuender Prüfer: Prof. Dr. Jochen Wittmann
Zweitgutachter: Prof. Dr. Zaphod Beeblebrox

Eingereicht am: 01.08.2026

Konstantin Lucius Regenhardt

Thema der Arbeit

Entwurf und Evaluation eines automatisierten Migrationswerkzeugs von Katalon-Groovy-Tests zu Selenium-Pytest

Stichworte

Kurzzusammenfassung

In der Softwareentwicklung bieten proprietäre Low-Code-Plattformen wie Katalon einen schnellen Einstieg in die Testautomatisierung. Mit steigender Projektkomplexität führen diese jedoch häufig zu einem massiven Kostenanstieg durch unflexible Lizenzmodelle für fortgeschrittene Funktionen. Da die Testlogik und Datenobjekte in proprietären Formaten gespeichert sind, stehen Nutzer vor dem Dilemma, entweder hohe Gebühren zu zahlen oder den Verlust der bisherigen Entwicklungsarbeit bei einem Plattformwechsel hinzunehmen. Ziel dieser Bachelorarbeit ist die Entwicklung eines programmatischen Migrationspfades von Katalon zu einer Open-Source-Alternative, geschrieben in Selenium. Dabei soll die manuelle Neuerstellung der Test-Suites durch eine automatisierte Transformations-Pipeline ersetzt werden, um die bisherige Arbeit nicht zu verlieren. Es wird ein Migrations-Algorithmus entworfen, der auf Regular Expressions und Python-Methoden basiert. Dieser scannt die proprietären Strukturen, übersetzt die bestehende Testlogik von Groovy nach Python und bildet die internen Datenobjekte sowie Referenzdateien auf eine neue, Open-Source Projektstruktur im Selenium-Framework ab. Das Ergebnis ist ein funktionsfähiger Prototyp eines Migrations-Tools. Dieses Tool ermöglicht den Transfer von Test-Suites in ein erweiterbares Open-Source-Projekt. Dabei bleibt die Integrität der Tests gewahrt und die Abhängigkeit von Lizenzgebühren wird vollständig eliminiert. Die Arbeit zeigt auf, wie durch automatisierte Code-Migration und -Transformation der Wechsel von proprietären „Low-Code“ zu Open-Source „Pro-Code“ gelingen kann. Die Ergebnisse bieten Nutzern eine gute Möglichkeit für den Ausstieg aus proprietären Test-Ökosystemen.

Konstantin Lucius Regenhardt

Title of Thesis

From Proprietary Test Automation to Open Execution: Design and Evaluation of a Katalon-to-Selenium/Pytest Migration Tool

Keywords

Abstract

In software development, proprietary low-code platforms like *Katalon* offer a fast initial entry point into test automation. However, as project complexity scales, these systems frequently lead to a massive escalation of costs due to inflexible licensing frameworks. This thesis addresses this „Vendor Lock-In“ problem by establishing an automated transformation pipeline that migrates test suites from closed Katalon structures into an open-source, flexible framework written in Python using *Selenium*.

A core migration algorithm based on Python execution logic and Regular Expressions (Regex) handles the automated parsing of proprietary formats, translations of test scripts from *Groovy* to *Python*, and mapping of object repositories. The evaluation showcases a fully functional prototype that successfully eliminates license fee dependencies while completely preserving test suite runtime integrity.

Abkürzungs- und Begriffsverzeichnis

API – Application Programming Interface: Eine definierte Schnittstelle, über die Softwarekomponenten miteinander kommunizieren. Selenium stellt eine API bereit, über die Tests den Browser steuern.

Build-Time: Der Prozessablauf während der Ausführung des Migrationswerkzeugs. Im Gegensatz zu Runtime (dem Zeitpunkt der Testausführung) finden alle Transformationen, Konvertierungen und Code-Generierung während der Build-Time statt. In diesem Projekt umfasst die Build-Time alle Pipeline-Module (test_suite_translator, test_transpiler, test_assembler, etc.), die ein Katalon-Projekt einmalig in ein Selenium/Pytest-Projekt konvertieren. 11, 12, 13, 20, 20, 20, 21

CI – Continuous Integration: Ein Softwareentwicklungsprinzip, bei dem Code-Änderungen regelmäßig in ein gemeinsames Repository integriert und automatisch gebaut und getestet werden. Automatisierte Testsuiten sind ein zentrales Werkzeug im CI-Prozess.

Compiler: Ein Programm, das Quellcode einer Programmiersprache analysiert und auf Fehler prüft oder in eine andere Form überführt. In dieser Arbeit bezeichnet Compiler den statischen Typ-Analysator Pyright, der Python-Dateien vor der Ausführung auf Typ- und Importfehler untersucht. 20

Pytest Fixture: Ein Mechanismus in Pytest, der wiederverwendbare Test-Vorbedingungen und -Nachbedingungen definiert. Fixtures werden in der Datei conftest.py deklariert und stehen allen Tests im Projekt automatisch zur Verfügung, ohne explizit importiert werden zu müssen. Typische Anwendungen sind das Erstellen eines WebDriver-Objekts oder das Einrichten von Testdaten. 9

Groovy: Eine dynamisch typisierte Skriptsprache, die auf der JVM läuft und vollständig mit Java interoperabel ist. Katalon Studio verwendet Groovy als Skriptsprache für Testskripte. In dieser Arbeit wird Groovy-Code mittels Regex-Transformation in Python übersetzt. 4, 6, 14

IDE – Integrated Development Environment: Eine Software-Anwendung, die Entwicklern eine umfassende Umgebung für die Softwareentwicklung bietet. Typischerweise umfasst sie einen Code-Editor, Debugger, Build-Werkzeuge und oft eine grafische Projektansicht. Katalon Studio ist eine solche IDE, die speziell auf Testautomatisierung ausgelegt ist. 5, 5, 5, 6, 11

JVM – Java Virtual Machine: Eine Laufzeitumgebung, die plattformunabhängige Ausführung von Java-Bytecode ermöglicht. Groovy, die Skriptsprache von Katalon Studio, läuft auf der JVM. 4

Locator: Ein Ausdruck, mit dem Selenium ein HTML-Element auf einer Webseite identifiziert. Gängige Locator-Strategien sind ID, CSS-Selektor, XPath und Name. Katalon speichert Locatoren im Object Repository; bei der Migration werden sie in Selenium-kompatible By-Strategien überführt.

Match: Eine erfolgreiche Übereinstimmung eines Regular-Expression-Musters mit einem Teil des Eingabetextes. Wenn ein Regex-Muster einen Textbereich findet, der der definierten Regel entspricht, wird dieser erkannte Bereich als Match bezeichnet. Ein Match enthält typischerweise auch Capture Groups, die Teilinformationen des Musters speichern. 14

Object Repository: Eine zentrale Datenstruktur in Katalon Studio, die alle Test Objects (z. B. Schaltflächen, Eingabefelder) mit ihren Lokalisierungsstrategien speichert. Jedes Objekt enthält Eigenschaften wie XPath, CSS-Selektor oder ID, anhand derer Selenium das Element im DOM der Webanwendung identifiziert. 4, 12

PoC – Proof of Concept: Ein Prototyp oder eine Machbarkeitsstudie, die zeigt, dass ein Konzept oder eine Idee praktisch umsetzbar ist. In dieser Arbeit bezeichnet PoC die erste Umsetzung eines Migrationsskripts zur Überprüfung der Machbarkeit.

Regex – Regular Expression: Ein formaler Ausdruck zur Beschreibung von Zeichenketten-Mustern. In dieser Arbeit werden Regular Expressions zur Transformation von Groovy-Syntax in Python-Syntax eingesetzt. VIII, 14, 14, 14, 18, 19, 21

Repository: Ein versioniertes Archiv für Quellcode und zugehörige Dateien, typischerweise verwaltet mit einem Versionskontrollsystem wie Git. In dieser Arbeit bezeichnet Repository sowohl das Quell-Katalon-Projekt als auch das generierte Ziel-Selenium/Pytest-Projekt als eigenständige, versionierbare Einheiten. 20

Runtime: Der Zeitpunkt der Ausführung der generierten Tests. Im Gegensatz zu Build-Time (während der Migration) werden während der Runtime die migrierten Selenium-Pytest-Tests tatsächlich ausgeführt. Die in src/runtime/ gespeicherten Dateien (base_test.py, katalon_helpers.py) werden während dieser Phase verwendet und sind nicht Bestandteil des Migrationsprozesses. 11, 13, 13, 20, 20

SUT – System Under Test: Das zu testende System — in dieser Arbeit die Webanwendung, auf die die automatisierten Selenium-Tests angewendet werden.

Test Object: Ein in Katalon Studio definiertes Objekt, das ein HTML-Element auf einer Webseite repräsentiert. Test Objects enthalten Informationen wie Name, Beschreibung und Locator. Sie werden im Object Repository gespeichert und in Testskripten referenziert. VI, 11, 12

Transpilation: Der Prozess der automatischen Übersetzung von Quellcode einer Programmiersprache in eine andere. Im Unterschied zur Kompilation bleibt das Abstraktionsniveau erhalten. In dieser Arbeit bezeichnet Transpilation die Übersetzung von Groovy-Testskripten nach Python.

UI – User Interface: Die Benutzerschnittstelle einer Anwendung, über die Nutzer mit dem System interagieren. Im Kontext dieser Arbeit bezeichnet UI die grafische Oberfläche, die durch automatisierte Tests validiert wird. 4, 5

Vendor Lock-in: Die technische oder wirtschaftliche Abhängigkeit eines Unternehmens von einem einzelnen Anbieter, die einen Wechsel zu Alternativen erschwert oder verteuert. Im Kontext dieser Arbeit bezeichnet Vendor Lock-in die Bindung an Katalon Studio durch proprietäre Dateiformate und lizenzpflichtige Funktionen. VI, 3, 3

WebDriver: Eine standardisierte API (W3C-Standard), über die Programme einen Webbrowser programmatisch steuern können. Selenium WebDriver ist die Referenzimplementierung und bildet die Grundlage aller in dieser Arbeit generierten Tests. 2

Inhaltsverzeichnis

Abkürzungs- und Begriffsverzeichnis IV

Abbildungsverzeichnis	VII
Tabellenverzeichnis	VIII
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Ziel & Aufbau der Arbeit	2
2 Grundlagen	2
2.1 Automatisiertes Testen	2
2.2 Low-Code-Plattformen im Testbereich	3
2.3 Vendor Lock-in bei proprietären Plattformen	3
3 Strukturanalyse	4
3.1 Katalon Projektstruktur	4
3.1.1 Elementare Ordner und Dateien	4
3.1.2 Verschachtelungen in der Katalon Struktur	5
3.2 Selenium/Pytest Projektstruktur	8
3.2.1 Testskripte & Variablen	8
3.2.2 Testdaten & Einbindung	8
3.2.3 Pytest Fixtures & Konfiguration	9
4 Konzeption der Migrationspipeline	11
4.1 Architekturüberblick	11
4.2 Transformationsablauf	12
4.2.1 Strukturelle Transformation: Projektlayout	12
4.2.2 Semantische Transformation: Katalon Test Object zu Selenium Locators ..	12
4.2.3 Syntaktische Transformation: Groovy zu Python	13
5 Implementierung	20
5.1 Projektstruktur des Migrators	20
5.1.1 Build-Time vs. Runtime	20
5.2 Pipeline-Module im Detail	21
6 Evaluation	21
6.1 Migrationsergebnis	22
6.2 Strukturelle Integrität	22
6.3 Codequalität: Vorher-Nachher-Vergleich	23
6.4 Aufwandsvergleich	24
7 Diskussion	24
8 Fazit & Ausblick	24
Quellenverzeichnis	25
Erklärung zur selbstständigen Bearbeitung	26

Abbildungsverzeichnis

Abbildung 1	Aktivitätsdiagramm der Transpilation: Transformationspfad eines Katalon-Tests zu einem Python-Pytest-Test.	14
Abbildung 2	Durch das <code>katalon_lines_pattern</code> -Muster werden aus einem Block aus Katalon Zeilen, drei getrennte Regex-Matches mit jeweils drei Gruppen für Klasse, Methode und Parameter extrahiert.	16
Abbildung 3	Das <code>fto_as_param_pat</code> -Muster erkennt in der Parsing-Phase die <code>findTestObject()</code> -Methode als ersten Parameter, falls ein Komma folgt und ermöglicht es diese zu trennen.	17
Abbildung 4	Das <code>fto_param_str_pattern</code> -Muster extrahiert die <code>findTestObject()</code> -Methode mit dem String-Argument, das für die Transformation benötigt wird.	18

Tabellenverzeichnis

Tabelle 1 Übersicht der Regular Expression (Regex) Muster im Transpilationsprozess . 19

Tabelle 2 Migrationsergebnis: Alle Komponenten des Katalon-Projekts wurden
vollständig überführt. 22

1 Einleitung

1.1 Motivation

Automatisiertes Testen ist eine zentrale Komponente moderner Software-Qualitätssicherung. In vielen Teams werden Testautomatisierungs-Frameworks eingesetzt, um Regressionstests zu beschleunigen und das Vertrauen in Veröffentlichungen zu stärken[1]. Anfangs nutzte unser Team Katalon Studio als IDE um neben der Entwicklung einer internen Geschäftssoftware auch eine grundlegende Testinfrastruktur aufzubauen. Im Laufe der Zeit wuchs die Infrastruktur stark an und umspannte den Großteil der Software.

Da sich die Anforderungen des Projekts weiterentwickelten, benötigte das Team fortgeschrittenere Automatisierungs-Techniken, wie automatisiertes Berechnen von Datums-Einträgen und eine tiefere Kontrolle über die Testausführung und das Debugging. In dieser Phase traten erste Blockaden auf: Erweiterte Funktionen, die im Programmieren als elementar gesehen werden, erforderten zusätzliche Lizenzen, was erhebliche Kosten verursachte und die Skalierung der Infrastruktur einschränkte. Dies kristallisierte sich schnell als ein blockierendes Problem heraus da die Preise in keinem Verhältnis zu den Leistungen standen und über längere Zeit nicht tragbar waren. Als Lösungsansatz wurde eine Migration zu einem Open-Source Projekt vorgeschlagen.

Angesichts der erheblichen bestehenden Investitionen in eine Katalon-basierte Test-Umgebung war ein vollständiges Neuschreiben der aller gesammelten Tests von Grund auf, nicht machbar. Deswegen wurde ein Migrationsansatz als PoC untersucht, um die zuvor erstellten Tests und alles Zugehörige zu erhalten und gleichzeitig den Übergang zu einer offenen und erweiterbaren Open-Source Struktur zu ermöglichen.

1.2 Problemstellung

Das Kernproblem das in dieser Arbeit behandelt wird, ist die Frage wie ein bestehendes Katalon-Projekt beim Wechsel in eine lizenzunabhängige Open-Source Umgebung beibehalten und wiederverwendet werden kann. Eine manuelle Migration von Katalon-Tests zu Selenium/Pytest ist extrem zeitaufwendig und bei einer großen Testmenge praktisch unerreichbar. Das liegt daran, dass neben der reinen Skriptübersetzung eine Menge an zusätzlichen Abhängigkeiten und Konfigurationen berücksichtigt werden müssen. Ein Katalon-Projekt besteht nicht nur aus Testskripten, sondern auch aus einer Vielzahl von Assets wie Objekt-Repositories, Variablen-Definitionen, globalen Profilen und Datenabhängigkeiten. Diese sind jeweils in proprietären Formaten gespeichert und müssten manuell „ausgegraben“ und zusammengeführt werden. Eine manuelle Migration würde daher nicht nur die Übersetzung von Groovy nach Python erfordern, sondern auch die Rekonstruktion der gesamten Projektstruktur und die Anpassung an die Open-Source-Frameworks.

Die technische Herausforderung liegt nicht nur in der syntaktischen Konvertierung des Testcodes, sondern in der End-to-End-Transformation eines Testautomatisierungsprojekts in ein ausführbares und skalierbares alternatives Format.

1.3 Ziel & Aufbau der Arbeit

Das Ziel dieser Arbeit ist es, einen automatisierten Migrator zu entwerfen, der Katalon-basierte Testprojekte in eine Python-Selenium/Pytest-Projektstruktur konvertiert.

Um dieses Ziel zu erreichen wird in dieser Arbeit zuerst die Struktur eines Katalon-Projektes und die eines Python-Selenium/Pytest-Projektes analysiert. Dann wird Stück für Stück ein Migrationsalgorithmus entworfen, der die proprietären Formate des einen, in offen nutzbare Formate des anderen überführt. Anschließend wird ein Prototyp implementiert, der die Transformation automatisiert und die Integrität der Tests sicherstellt. Abschließend werden die Ergebnisse evaluiert und diskutiert, um die Effektivität des Migrationsprozesses zu bewerten.

2 Grundlagen

2.1 Automatisiertes Testen

Softwaretests sind ein grundlegender Bestandteil der Qualitätssicherung in der Softwareentwicklung. Ihr Ziel ist es, sicherzustellen, dass ein System korrekt funktioniert und definierte Anforderungen erfüllt. Dabei unterscheidet man grob zwischen manuellen und automatisierten Tests.

Bei manuellen Tests, auch „Regressionstests“ genannt, führt eine Person die Testschritte selbst aus und bewertet das Ergebnis. Dieser Ansatz ist bei kleinen, seltenen oder exemplarischen Überprüfungen gut nutzbar, jedoch skaliert dessen Zeitaufwand schnell. Mit wachsender Softwarekomplexität steigt der Zeitaufwand für manuelle Tests exponentiell.

Automatisiertes Testen bezeichnet das Ausführen von Tests mit Hilfe von Testskripten, die von einem Computer ausgeführt werden. Die Skripte definieren Eingaben, Aktionen und erwartete Ergebnisse. Wesentliche Vorteile automatisierter Tests sind[2]:

- **Wiederholbarkeit:** Dieselben Tests können beliebig oft und konsistent ausgeführt werden.
- **Geschwindigkeit:** Große Test-Suits lassen sich in kurzer Zeit ausführen.
- **Regressionssicherheit:** Nach Codeänderungen kann schnell geprüft werden, ob bestehende Funktionalität noch korrekt arbeitet.
- **CI/CD-Integration:** Automatisierte Tests können direkt in Deployment-Pipelines eingebunden werden.

Ein häufig eingesetztes Verfahren ist der **End-to-End-Test** (E2E-Test), bei dem eine Anwendung aus Nutzerperspektive durch die Benutzeroberfläche getestet wird. Werkzeuge wie die Selenium WebDriver ermöglichen solche Tests für webbasierte Anwendungen, indem sie einen Browser programmatisch steuern.

2.2 Low-Code-Plattformen im Testbereich

Low-Code-Plattformen sind Entwicklungsumgebungen, die komplexe technische Operationen durch grafische Oberflächen und vorgefertigte Bausteine abstrahieren. Im Bereich der Testautomatisierung bieten sie einen niedrighschwelligen Einstieg: Tests können über eine GUI aufgezeichnet oder aus einem Katalog von vordefinierten Aktionen zusammengestellt werden, ohne tiefgehende Programmierkenntnisse vorauszusetzen.

Diese Eigenschaft macht Low-Code-Tools attraktiv für Teams die wenig Programmierkenntnisse haben. Mit wachsender Anforderungskomplexität stoßen auch sie oft schnell an Grenzen[3]:

- Individuelle Logik, die über vorgefertigte Bausteine hinausgeht, ist schwer oder gar nicht umsetzbar.
- Die Testlogik ist eng an die Plattform gebunden und schlecht in externe Versionskontrollsysteme integrierbar.
- Skalierbarkeit und Anpassbarkeit sind durch das Plattformmodell begrenzt.

Im Gegensatz dazu stehen sogenannte **Pro-Code**-Ansätze, bei denen Tests vollständig in einer allgemeinen Programmiersprache wie Python geschrieben werden. Frameworks wie Selenium und Pytest bieten dabei maximale Flexibilität, erfordern aber entsprechende Programmierkenntnisse. Dabei sind diese Frameworks alle Open-Source, also kostenlos, öffentlich zugänglich und transparent. Der Übergang von Low-Code zu Pro-Code ist inhaltlich des Kerns dieser Arbeit.

2.3 Vendor Lock-in bei proprietären Plattformen

Vendor Lock-in beschreibt die Abhängigkeit eines Nutzers oder einer Organisation von einem bestimmten Anbieter, sodass ein Wechsel zu einer Alternative mit erheblichem Aufwand oder hohen Kosten verbunden ist[4]. Diese Abhängigkeit entsteht häufig durch proprietäre Dateiformate, plattformspezifische Programmiersprachen oder APIs, die außerhalb des jeweiligen Ökosystems nicht verwendet werden können.

Im Bereich der Testautomatisierung zeigt sich Vendor Lock-in beispielsweise dann, wenn:

- Testskripte in einer plattformeigenen Skriptsprache verfasst sind und nicht ohne Weiteres auf andere Frameworks übertragen werden können,
- interne Datenstrukturen wie Objekt-Repositorys oder Profil-Konfigurationen in proprietären Formaten gespeichert sind,
- bestimmte Funktionen nur in kostenpflichtigen Lizenzstufen verfügbar sind.

Die Konsequenz ist, dass Teams entweder rasant steigende Lizenzkosten akzeptieren müssen oder bei einem Plattformwechsel einen erheblichen Teil ihrer bisherigen Arbeit verlieren. Für das in dieser Arbeit beschriebene Projekt manifestierte sich dieses Problem in Form von grundlegenden Entwicklungsfunktionen, wie das Debugging über die CLI, die teurere Lizenzpakete erforderten, was wiederum die langfristige Nutzung der Plattform sehr teuer machte.

3 Strukturanalyse

3.1 Katalon Projektstruktur

Ein Katalon Studio Projekt besteht, aus dem Blickwinkel des IDE-Benutzers aus Tests, einem „Object Repository“, globalen Variablen in Profilen, testeigenen Variablen und eingebundenen Test-Daten. Diese werden in einer hierarchischen Ordnerstruktur gespeichert, die von der IDE verwaltet wird.

3.1.1 Elementare Ordner und Dateien

3.1.1.1 Test Cases & Variablen

Die Testskripte in Katalon sind in Groovy geschrieben, einer dynamischen Sprache, die auf der Java Virtual Machine (JVM) läuft. Sie werden mit einer Vielzahl von eingebauten Keywords und Funktionen geliefert, die speziell für Testautomatisierung entwickelt wurden. Variablen können sowohl global als auch lokal definiert werden, wobei globale Variablen in Profilen gespeichert sind und in allen Tests zugänglich sind.

3.1.1.2 Object Repository

Das Object Repository ist ein zentrales Element in der Katalon Umgebung, das das Speichern von „User Interface (UI)“-Elementen ermöglicht. Es speichert die Eigenschaften von HTML-Elementen, die während der Testausführung verwendet werden. Dafür kann der Benutzer die Elemente mit einem Tool das Katalon Studio bereitstellt auf der Webseite auswählen und im Object Repository generieren lassen. Diese Elemente können in verschiedenen Tests wiederverwendet werden, was die Wartung und Skalierbarkeit der Tests erleichtert.

3.1.1.3 Globale Variablen & Profile

Die User können in Katalon eigene globale Variablen definieren, die in verschiedenen Test Cases verwendet werden können. Diese Variablen werden in Profilen gespeichert, welche unterschiedliche Benutzende oder Umgebungen repräsentieren können. Profile ermöglichen es, Tests in verschiedenen Konfigurationen auszuführen, ohne den Testcode selbst ändern zu müssen. Hat ein Profil zum Beispiel die Variable „URL“ mit dem Wert `https://staging.example.com` und ein anderes Profil die gleiche Variable mit dem Wert `https://production.example.com`, kann der Test in beiden Umgebungen ausgeführt werden, indem einfach das entsprechende Profil ausgewählt wird.

3.1.1.4 Testdaten & Einbindung

Katalon kann Testdaten aus verschiedenen Quellen, wie Excel-Dateien, CSV-Dateien oder Datenbanken einbinden. Diese Testdaten können in den Testskripten referenziert werden, um die Tests mit unterschiedlichen Eingabewerten auszuführen.

3.1.1.5 Custom Keywords

Benutzer/Innen können eigene Methoden, sogenannte „Custom Keywords“ erstellen, die in mehreren Test Cases wiederverwendet werden können. Diese Keywords werden in Groovy geschrieben und ermöglichen es, komplexere Logik für Tests zu nutzen und diese wiederverwendbar zu machen.

3.1.2 Verschachtelungen in der Katalon Struktur

Alle Dateien und Ordner die man in der Katalon Integrated Development Environment (IDE) erstellt, werden durch den Viewport übersichtlich dargestellt. Sobald man sich die Ordnerstruktur eines Katalon-Projekts jedoch außerhalb der IDE anschaut, erkennt man, dass die Testskripte nicht nur eine einzelne Datei sind, wie im Programmieren üblich.

3.1.2.1 Verschachtelung der Test Cases

Ein „Test Case“ besteht in Katalon aus mehreren Dateien, die zusammenarbeiten. Folgt man der Katalon-Ordnerstruktur im Explorer unter „Test Cases“, so wie sie in der IDE zu sehen ist landet man bei einer .tc-Datei. Öffnet man diese, erkennt man schnell, dass es sich dabei um eine umbenannte .xml-Datei handelt. Sie enthält Meta-Daten, wie eine Beschreibung, den Testnamen, Tags, Kommentare, eine GUID und die eigentlichen Werte der Test-eigenen Variablen. Es gibt keine offensichtliche Referenz auf die eigentliche Testlogik. Diese ist in einer weiteren Datei gespeichert. Um sie zu finden muss man den Ordner „Scripts“ öffnen, dann den Pfad des „Test Cases“ spiegeln. Dort befindet sich ein Ordner der den Testnamen trägt und dieser enthält dann eine .groovy-Datei, die den Namen „Script“ kombiniert mit einer zufälligen Nummer trägt. Die Nummer steht in keiner Verbindung zu der .tc-Datei. In dieser .groovy-Datei befindet sich dann die eigentliche Testlogik. Greift der Test auf eine Variable zu, wird im Skript der „rohe“ Name der Variable ausgeschrieben. „Roh“ bedeutet, dass der Name im Code keinen Datentyp hat und auf nichts im Test verweist. Die Variable wird im Test weder deklariert, noch initialisiert.

3.1.2.2 Verschachtelung der Globalen Variablen

Wenn der Test eine Globale Variable verwendet, zeigt sich dies im Code als „GlobalVariable.VARIABLENNAME“. Der Wert der Variable ist nicht im Skript zu finden sondern in der Profil-Datei mit der Endung .gbl, die im Ordner „Profiles“ gespeichert ist. Diese Datei ist ebenfalls eine .xml-Datei und enthält die Variablen als Eintrag in einer „GlobalVariableEntity“ - ein Eintrag der die Variable beschreibt. Diese Einträge enthalten den tatsächlichen Wert und Meta-Daten, bestehend aus einer Beschreibung, dem Variablennamen, dem Datentyp und einem Boolean der beschreibt ob der Wert „protected“ ist. Ist er „protected“, wird der Wert als sensibel behandelt und in der UI und bei der Bearbeitung nur mit dem Stern-Charakter maskiert angezeigt.

3.1.2.3 Verschachtelung der Testdaten

Sobald man mit Katalon Studio ein Dateiformat mit Daten einbinden möchte, erstellt Katalon Studio eine .dat-Datei im Ordner „Data Files“ die den gleichen Namen wie die eingebundene Datei trägt. Die Datei enthält ebenfalls XML-Strukturen, nur ein paar Meta-Daten enthalten. Unter anderem den Pfad zur eingebundenen Datei, die Dateiarart, ggf. die Trennzeichen und ob der Pfad Projektintern oder -extern ist. Die eigentlichen Daten sind nicht in der .dat-Datei enthalten, sondern in der eingebundenen Datei.

3.1.2.4 Verschachtelung der Custom Keywords

Der Großteil der „Custom Keywords“-Logik wird im Ordner „Keywords“ und selbst erstellten Unterordnern gespeichert, und ist in Groovy in der Form „KlassenName.groovy“ geschrieben. Diese Skripte enthalten die eigentlichen Klassen und Methoden die in den Tests aufgerufen werden. Im Ordner „Libs“ gibt es die „CustomKeywords.groovy“-Datei. Diese definiert statische Weiterleitungen mit dem gleichen Namen wie die Klassen im Ordner „Keywords“. Diese Methoden rufen die eigentlichen Methoden in den Klassen auf und machen sie dadurch in jedem Test aufrufbar.

Die folgende Abbildung zeigt die vollständige Dateistruktur des Katalon-Projekts, wie sie außerhalb der IDE im Filesystem vorliegt — inklusive aller in den vorangehenden Abschnitten beschriebenen Verschachtelungen:

```

sample-website-katalon-tests/
|
|— Test Cases/                                - Katalon IDE zeigt diese Ansicht
|   |— Users/
|   |   |— filter_for_admins.tc                - .tc = XML Meta-Datei (Beschreibung, Variable)
|   |   |— search_for_david_kim.tc            - KEINE Testlogik enthalten!
|   |   |— show_active_viewers.tc
|   |   |— Menus/
|   |       |— navigation_bar.tc
|   |
|   |— Scripts/                                - Echte Testlogik (außer IDE verborgen)
|   |   |— Users/
|   |   |   |— filter_for_admins/
|   |   |       |— Script1781348815820.groovy - Zufällige Nummer! (!)
|   |   |   |— search_for_david_kim/
|   |   |       |— Script1781447000826.groovy - Entspricht search_for_david_kim.tc
|   |   |   |— show_active_viewers/
|   |   |       |— Script...groovy
|   |   |— Menus/
|   |       |— navigation_bar/
|   |           |— Script...groovy
|   |
|   |— Object Repository/                      - Lokalisierungsstrategien (Selektoren)
|   |   |— All_Users/
|   |   |   |— view_all_users_btn.rs          - .rs = XML Objekt (XPath, CSS, ID...)
|   |   |   |— search_input.rs
|   |   |   |— roles_dropdown.rs
|   |   |   |— status_dropdown.rs
|   |   |   |— user_row.rs
|   |   |   |— user_row_view_btn.rs
|   |   |— Nav_Bar/
|   |       |— ...weitere Test Objects
|   |
|   |— Profiles/                              - Globale Variablen nach Umgebung
|   |   |— default.glbl                       - .glbl = XML (GlobalVariableEntity Einträge)
|   |
|   |— Data Files/                            - Externe Testdaten-Referenzen
|   |   |— users.dat                          - .dat = XML Meta (verweist auf echte Datei)
|   |
|   |— Keywords/                              - Custom Methoden/Helper
|   |   |— data/
|   |   |— TESTER.groovy                     - Groovy Klasse mit eigenen Methoden
|   |
|   |— Libs/
|   |   |— CustomKeywords.groovy             - Statische Weiterleitungen zu Keywords
|   |
|   |— Plugins/, Test Listeners/, ...

```

Listing 1: Katalon-Projektstruktur: Komplexe Verschachtelung und Format-Vielfalt (.tc, .groovy, .rs, .glbl, .dat, .xml)

Diese Struktur verdeutlicht die Komplexität, die eine Migration bewältigen muss. Besonders auffällig ist die Vielfalt der Dateiformate: Obwohl Dateien wie .tc, .rs, .glbl und .dat alle auf

XML basieren, tragen sie proprietäre Endungen, die ihren Inhalt verschleiern. Hinzu kommt, dass ein einziger Test aus zwei getrennten Dateien in zwei verschiedenen Ordnern besteht — die Metadaten liegen in Test Cases/, die eigentliche Testlogik in Scripts/. Die Verbindung zwischen ihnen ist dabei nur indirekt über den Pfad herstellbar: Scripts/Users/filter_for_admins/Script1781348815820.groovy gehört zu Test Cases/Users/filter_for_admins.tc. Erschwerend kommt hinzu, dass die Script-Dateien zufällig generierte Nummern im Namen tragen, die keinerlei Rückschlüsse auf den zugehörigen Test Case erlauben.

3.2 Selenium/Pytest Projektstruktur

Ein Projekt, das Selenium und Pytest für die Testautomatisierung nutzt, ist im Gegensatz zu Katalon nicht an eine proprietäre Projektstruktur gebunden. Pytest erwartet nur eine nachvollziehbare Dateiorganisation durch Konventionen, gültige Python-Module und eine Konfiguration im Projektwurzelverzeichnis, der „root“, in Form einer pyproject.toml oder pytest.ini[5]. In einem reinen Testprojekt enthält das Repository dabei keinen Anwendungscode, sondern ausschließlich Testfälle, gemeinsam genutzte Hilfsmodule, Konfigurationsdateien und Testdaten. Typischerweise existiert ein zentrales „tests“-Verzeichnis, das bei wachsender Projektgröße weiter in Teilbereiche der zu testenden Anwendung gegliedert wird, beispielsweise in UI-Tests und API-Tests[6]. Das aus Python-Paketprojekten bekannte src-Layout ist in diesem Fall nicht zwingend erforderlich. Laut Python Packaging User Guide dient das src-Layout vor allem dazu, importierbaren Anwendungscode klar von der Projekt-root zu trennen[7].

3.2.1 Testskripte & Variablen

Die eigentlichen Testfälle folgen den von Pytest vorgesehenen Namenskonventionen wie test_NAME.py oder NAME_test.py[6]. Anders als bei Katalon besteht ein Testfall dabei in der Regel aus genau einer Python-Datei, in der die Testlogik direkt lesbar ist. Wiederverwendbare Browser-Interaktionen, Element-Selektoren oder Methoden können in separate Hilfsmodule ausgelagert werden. Diese heißen oft „Helper“- oder „Utility“-Funktionen.

Variablen werden in einem solchen Projekt nicht über proprietäre Profil- oder Metadateien verwaltet, sondern über normale Python-Programmierstrukturen. Typisch sind Parametrisierungen in den Testfunktionen, Rückgabewerte von Fixtures, Umgebungsvariablen oder externe Konfigurationsdateien. Dadurch bleibt nachvollziehbar, woher ein Wert stammt und an welcher Stelle er in den Test eingebunden wird.

3.2.2 Testdaten & Einbindung

Testdaten werden häufig in einem eigenen Verzeichnis wie z. B. „data“, „resources“ oder „testdata“ abgelegt und dann bei Bedarf direkt von den Tests oder von Hilfsmodulen eingelesen. Dabei kann es sich beispielsweise um JSON-, CSV-, XML- oder Excel-Dateien handeln, jedoch ist praktisch jedes Format nutzbar. Im Unterschied zu Katalon entsteht hierbei keine zusätzliche proprietäre Metadatei, die nur einen Verweis auf die eigentliche Datenquelle enthält. Stattdessen sind Dateipfade, Ladeverhalten und Weiterverarbeitung unmittelbar im Python-Code oder in den zugehörigen Hilfsfunktionen sichtbar.

3.2.3 Pytest Fixtures & Konfiguration

Eine zentrale Rolle in Pytest nimmt die Datei `conftest.py` ein. Die dort definierten Pytest Fixture sind wiederverwendbare Testkomponenten und stehen allen Tests automatisch zur Verfügung und müssen nicht explizit importiert werden[8]. Auf diese Weise lassen sich gemeinsam genutzte Setups, etwa die Erstellung eines WebDriver-Objekts, Logins, temporäre Verzeichnisse oder standardisierte Testdaten, an einer gemeinsamen Stelle sammeln.

Die allgemeine Projektkonfiguration liegt typischerweise im root-Verzeichnis des Repositories. Dort können unter anderem Testpfade, Standardoptionen für den Testlauf oder Marker definiert werden[5].

Für ein reines Selenium/Pytest-Testprojekt ergibt sich damit eine vergleichsweise schlanke Struktur aus Testskripten, gemeinsam genutzten Fixtures, optionalen Hilfsmodulen und separaten Testdaten. Im Vergleich zur Katalon-Struktur sind sowohl die Pfade als auch die Abhängigkeiten zwischen Testlogik, Konfiguration und Daten deutlich direkter ablesbar.

Die folgende Abbildung zeigt die vom Migrator tatsächlich generierte Zielstruktur:

sample-website-selenium-tests/	- Neues Ziel-Projekt
─ pytest.ini	- Pytest-Konfiguration (generiert)
─ requirements.txt	- Python-Abhängigkeiten (generiert)
─ .gitignore	- Git-Konfiguration (generiert)
─ README.md	- Projektdokumentation (generiert)
─ data/	- Testdaten (aus Katalon "Data Files")
─ users.csv	- Originaldatei direkt kopiert
─ src/	
─ __init__.py	
─ tests/	- Aus: Test Cases/ + Scripts/
─ Users/	
─ test_filter_for_admins.py	- ../filter_for_admins/Script...groovy
─ test_search_for_david_kim.py	
─ test_show_active_viewers.py	
─ __init__.py	
─ Menus/	
─ test_navigation_bar.py	
─ __init__.py	
─ __init__.py	
─ object_repository/	- Aus: Object Repository/ (.rs XML → JSON)
─ All_Users/	
─ view_all_users_btn.json	- Inhalt: JSON (konvertiert aus .rs XML)
─ search_input.json	
─ roles_dropdown.json	
─ __init__.py	
─ Nav_Bar/	
─ __init__.py	
─ profiles/	- Aus: Profiles/ (.global XML)
─ global_variables.py	- Python-Klasse mit allen GlobalVariables
─ __init__.py	
─ variables/	- Aus: Test Cases/ (testeigene Variablen)
─ Users/	
─ var_search_for_david_kim.py	- Python-Klasse pro Test (wenn vorhanden)
─ __init__.py	
─ __init__.py	
─ runtime/	- Kopiert aus dem Migrator selbst
─ base_test.py	- Selenium WebDriver Base Class
─ katalon_helpers.py	- Kompatibilitäts-Helfer
─ __init__.py	

Listing 2: Tatsächlich generierte Selenium/Pytest-Projektstruktur nach Ausführung des Migrators.

4 Konzeption der Migrationspipeline

Eine ursprüngliche Idee, als die Aufgabe der Migration in ein anderes Ökosystem aufkam, war es ein Selenium/Pytest-Projekt zu schreiben, das Katalon-Tests direkt lesen und ausführen kann. Diese Idee wurde nach Rücksprache mit anderen Entwicklern jedoch schnell verworfen, da die proprietären Formate und die Katalon-spezifische Logik zu komplex waren, um sie direkt in einem Open-Source-Framework auszuführen. Stattdessen wurde ein Ansatz gewählt, der die Katalon-Testlogik in eine neue, offene Struktur transformiert. Durch diese komplette Trennung von Katalon-Strukturen konnten Wartbarkeit und Erweiterbarkeit gesichert werden. Für den nächsten Ansatz wurde erst eine kleine Pipeline gebaut, die versuchte, häufig vorkommende Methoden zu übersetzen. Dieser Ansatz zeigte sehr schnell, dass die Katalon-Struktur im Hintergrund der IDE viel komplexer und verschachtelter ist, als es auf den ersten Blick scheint. Mit der Zeit entwickelten sich erst Regex-basierte Methoden zum Erkennen aller Katalon Funktions-Aufrufe, dann Weitere zum Erkennen von Pfaden zu Test Objects, dann zum Erkennen von Variablen und schließlich zum Erkennen von Globalen Variablen und deren Werten. Mit der Zeit entstand so eine großläufige Migrationspipeline, die die zentralen Aspekte einer Katalon-Struktur erkennt, diese Transformiert und in einer neuen Selenium/Pytest-Struktur abspeichert.

4.1 Architekturüberblick

Die Migrationspipeline enthält viele kleine Module die zusammen Schritt für Schritt alle Inhalte transformieren. Zusammen ergeben sich fünf Kernfunktionen, die zusammen den kompletten Transformationsprozess abbilden. Der „Scanner & Parsing“-Schritt fungiert als Einstiegspunkt. Dabei scannt und extrahiert man alle Projektkomponenten wie Test Cases, Object Repository, Variablen und Testdaten systematisch aus dem alten Projekt. Dann der „Transpilations“-Schritt, mit der Aufgabe Regex-basierte Leseregeln auf den Groovy-Syntax anzuwenden und den ursprünglichen Code schrittweise zu erkennen und zu übersetzen. Eng verzahnt damit, passiert währenddessen das Mapping und die Validation, um Katalon-Datentypen auf ihre Python-Äquivalente abzubilden und gleichzeitig die Integrität aller Transformationen zu validieren. Danach folgt die Code-Generierung. Hier wird aus den transformierten Strukturen, gültiger Selenium-Pytest-Code der unmittelbar ausführbar ist generiert. Als letztes kommt der kombinierende zusammenführende Schritt, der alle generierten Dateien in eine korrekte und wartbare Selenium/Pytest-Projektstruktur schreibt. Diese Pipeline-Architektur gewährleistet, dass alle wichtigen Element eines Katalon-Projektes systematisch, nachvollziehbar und wartbar transformiert werden.

Die Implementierung des Migrations-Werkzeugs erfolgte in Python und nutzt eine streng modularisierte Architektur, die zwischen Build-Time (Migrations-Pipeline) und Runtime (Ziel-Projektdateien) unterscheidet.

4.2 Transformationsablauf

4.2.1 Strukturelle Transformation: Projektlayout

Wie in Listing 1 und Listing 2 (Kapitel 3) gezeigt, wird die hierarchische Katalon-Struktur in eine flache, einfach ausführbare Python-Projektstruktur überführt. Die wesentlichen Abbildungen sind:

- Test Cases/ & Scripts/ → src/tests/ (zusammengeführt zu je einer Datei pro Test)
- Object Repository/ → src/object_repository/ (XML-Inhalt zu JSON konvertiert)
- Profiles/ → src/profiles/global_variables.py (Python-Klasse mit allen Variablen)
- Data Files/ → data/ (Originaldatei direkt kopiert, proprietäre .dat-Metadatei fällt dabei weg)

4.2.2 Semantische Transformation: Katalon Test Object zu Selenium Locators

Das Katalon Object Repository speichert Testobjekte als .rs-Dateien — umbenannte XML-Dateien mit einer `WebElementEntity`-Struktur. Jede Datei beschreibt ein HTML-Element über eine `selectorCollection`, die eine oder mehrere Lokalisierungsstrategien als Schlüssel-Wert-Paare enthält (z. B. BASIC/XPath oder CSS), sowie ein `selectorMethod`-Feld, das die bevorzugte Strategie bestimmt.

Während der Build-Time konvertiert `object_repo_converter.py` diese XML-Dateien in das JSON-Format. Die interne Struktur bleibt dabei vollständig erhalten — lediglich das Dateiformat ändert sich. Das folgende Beispiel zeigt die Transformation der Objektdatei `view_all_users_btn`:

```
<!-- Object Repository/All_Users/view_all_users_btn.rs (Katalon XML) -->
<WebElementEntity>
  <name>view_all_users_btn</name>
  <selectorCollection>
    <entry>
      <key>BASIC</key>
      <value>//*[@id = 'hero-cta']</value>
    </entry>
  </selectorCollection>
  <selectorMethod>BASIC</selectorMethod>
</WebElementEntity>

// src/object_repository/All_Users/view_all_users_btn.json (generiertes JSON)
{
  "WebElementEntity": {
    "name": "view_all_users_btn",
    "selectorCollection": {
      "entry": { "key": "BASIC", "value": "//*[@id = 'hero-cta']" }
    },
    "selectorMethod": "BASIC"
  }
}
```

Zur Runtime liest die Hilfsfunktion `find_katalon_test_object()` in `katalon_helpers.py` die JSON-Datei ein, liest `selectorMethod` aus und sucht den passenden Wert aus der `selectorCollection`. Anschließend lokalisiert sie das Element über die Selenium-API: bei CSS mit `By.CSS_SELECTOR`, bei allen anderen Strategien (einschließlich BASIC) mit `By.XPATH`. Der Aufruf im generierten Testcode bleibt dabei strukturell identisch zum Katalon-Original:

```
// Katalon (Groovy):
WebUI.click(findTestObject('All_Users/view_all_users_btn'))

# Generierter Python-Code:
kh.find_katalon_test_object(self.driver, 'All_Users/view_all_users_btn').click()
```

Durch diese Zweistufigkeit — Format-Konvertierung zur Build-Time, Auflösung zur Runtime — muss der generierte Testcode selbst keine Kenntnis über die verwendete Lokalisierungsstrategie haben. Die Bindung zwischen Testlogik und Element-Selektor bleibt erhalten, ohne dass sich die Testeingabe ändert.

4.2.3 Syntaktische Transformation: Groovy zu Python

Die Groovy-Syntax wird durch Regex-Pattern-Matching in Python-Syntax überführt. Aus der Implementierung ergeben sich beispielsweise folgende Transformationen:

Katalon (Groovy):

```
WebUI.verifyTextPresent('View All Users', false)
WebUI.click(findTestObject('All_Users/view_all_users_btn'))
WebUI.setText(findTestObject('All_Users/search_input'), david)
WebUI.verifyTextPresent(GlobalVariable.user4name, false)
```

Generierter Python-Code:

```
assert 'View All Users' in self.driver.page_source
kh.find_katalon_test_object(self.driver, 'All_Users/view_all_users_btn').click()
kh.find_katalon_test_object(self.driver, 'All_Users/search_input').clear()
kh.find_katalon_test_object(self.driver, 'All_Users/
search_input').send_keys(vars.david)
assert GlobalVariable.USER4NAME in self.driver.page_source
```

Das Folgende Diagramm stellt diesen Ablauf detaillierter dar:

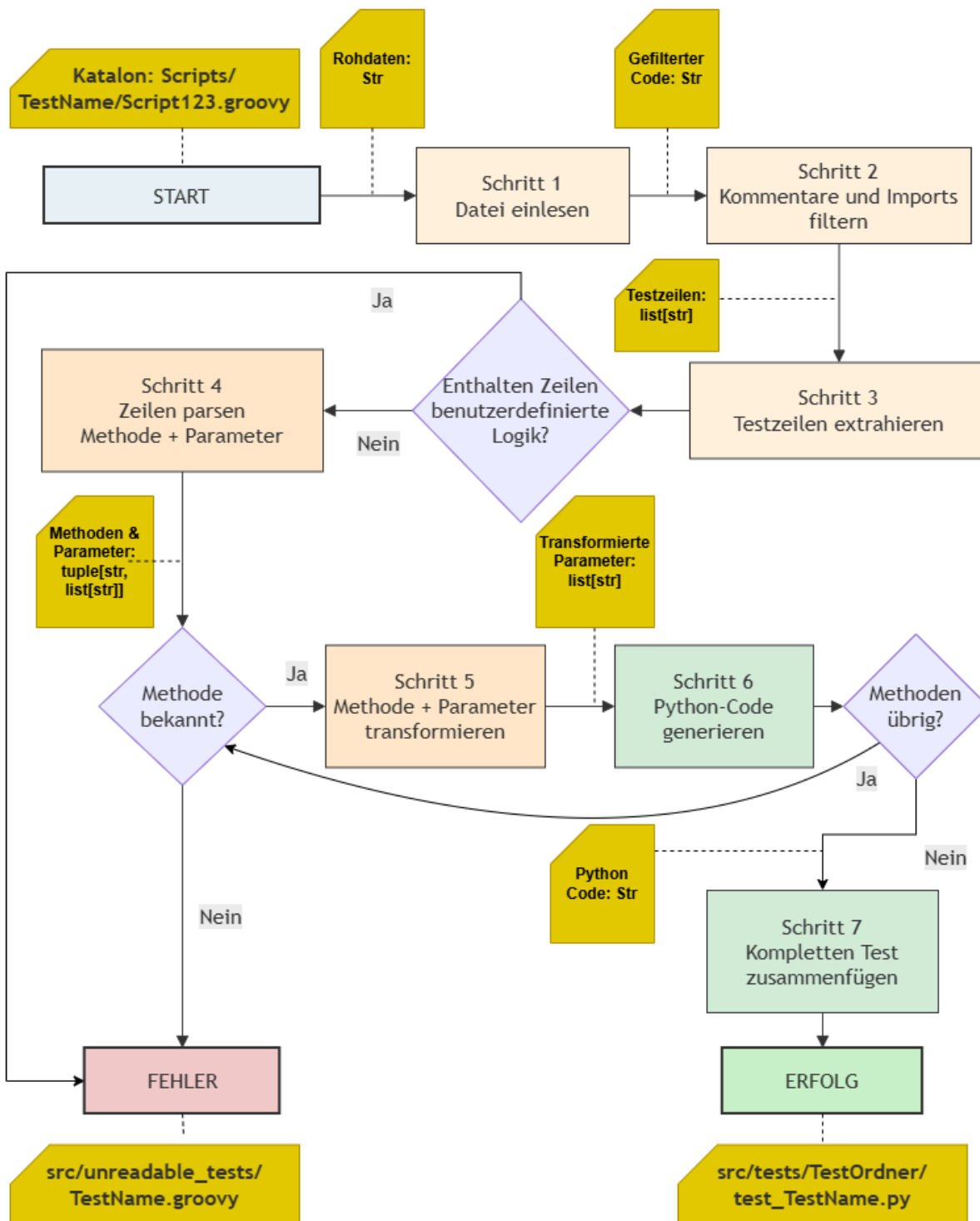


Abbildung 1: Aktivitätsdiagramm der Transpilation: Transformationspfad eines Katalon-Tests zu einem Python-Pytest-Test.

4.2.3.1 Regex-basierte Transformation

Der Migrator nutzt „Regular Expression (Regex)“ zum Erkennen von „Groovy“-Syntax. Nach Friedl wird Regex als „*a language used to describe patterns within text*“, definiert, also einer Sprache mit der Muster in einem Text beschrieben werden können[9]. Ein Match ist dabei eine erfolgreiche Übereinstimmung: Wenn das Muster einen Teil des Textes findet, der der Regel entspricht, wird dieser als Match bezeichnet. Im Kontext dieser Migration dienen Regex Muster

dazu, Groovy-Code zu identifizieren und ihn in seine Einzelteile zu zerlegen um diese wieder in Python-Äquivalente zu übersetzen.

Ein vereinfachtes Beispiel: Um alle Aufrufe der Katalon-Methode `webUI.click(...)` zu erkennen, wird ein Muster wie folgt definiert:

Muster: `WebUI\.click\((.*?)\)`

Ersatz: `{$1}.click()`

Dieser Ausdruck sucht nach dem Text „WebUI.click“, gefolgt von beliebigem Inhalt in Klammern, und ersetzt ihn durch den Inhalt der Klammern mit der Python-Methode `.click()`. Dadurch wird `WebUI.click(findTestObject('btn'))` zu `{find_katalon_test_object('btn')}.click()` transformiert.

Zur Veranschaulichung folgen Bilder, die den Ablauf des Parsens der „findTestObject“-Methode zeigen. Dabei wird regex101.com verwendet, um die Muster anschaulicher darzustellen.

Regular Expression

Matches · 3

:

/

(\w+)\.(\w+)\((.*)\)

Test String

WebUI.verifyTextPresent('View*All*Users',*false)

WebUI.click(findTestObject('All_Users/view_all_users_btn'))

WebUI.setText(findTestObject('All_Users/search_input'),*david)

Explanation

1st Capturing Group (\w+)

\w+ matches any word character (equivalent to [a-zA-Z0-9_])

+ matches the previous token one or more times, as many times as possible, giving characters back as needed (greedy)

\. matches the character . with numeric value 46₁₀ (56₈ or 2E₁₆) literally

2nd Capturing Group (\w+)

\w+ matches any word character (equivalent to [a-zA-Z0-9_])

+ matches the previous token one or more times, as many times as possible, giving characters back as needed (greedy)

\(matches the character (with numeric value 40₁₀ (50₈ or 28₁₆) literally

Match Information

Match 1	0-48	WebUI.verifyTextPresent('View*All*Users',*false)
Group 1	0-5	WebUI
Group 2	6-23	verifyTextPresent
Group 3	24-47	'View*All*Users',*false
Match 2	50-109	WebUI.click(findTestObject('All_Users/view_all_users_btn'))
Group 1	50-55	WebUI
Group 2	56-61	click
Group 3	62-108	findTestObject('All_Users/view_all_users_btn')
Match 3	111-173	WebUI.setText(findTestObject('All_Users/search_input'),*david)

Abbildung 2: Durch das katalon_lines_pattern-Muster werden aus einem Block aus Katalon Zeilen, drei getrennte Regex-Matches mit jeweils drei Gruppen für Klasse, Methode und Parameter extrahiert.

Regular Expression
Matches · 1
Steps

: / (findTestObject\(.*\)\)(?=?,)

Test String

```

WebUI.verifyTextPresent('View-All-Users', *false)
WebUI.click(findTestObject('All_Users/view_all_users_btn'))
WebUI.setText(findTestObject('All_Users/search_input'), *david)

```

Explanation

- 1st Capturing Group (findTestObject\(.*\))
 - findTestObject matches the characters findTestObject literally (case sensitive)
 - \ matches the character (with numeric value 40₁₀ (50₈ or 28₁₆) literally
 - .* matches any character (except for line terminators)
 - * matches the previous token zero or more times, as many times as possible, giving characters back as needed (greedy)
 - \ matches the character) with numeric value 41₁₀ (51₈ or 29₁₆) literally
- Positive Lookahead (?=,)
 - , matches the character , with numeric value 44₁₀ (54₈ or 2C₁₆) literally

Match Information

Match 1	125-165	findTestObject('All_Users/search_input')
Group 1	125-165	findTestObject('All_Users/search_input')

Abbildung 3: Das fto_as_param_pat-Muster erkennt in der Parsing-Phase die findTestObject()-Methode als ersten Parameter, falls ein Komma folgt und ermöglicht es diese zu trennen.

Regular Expression

Matches

```

: / (findTestObject\(((?=.*+').*\))

```

Test String

```

WebUI.verifyTextPresent('View-All-Users', 'false')
WebUI.click(findTestObject('All_Users/view_all_users_btn'))
WebUI.setText(findTestObject('All_Users/search_input'), 'david')

```

Explanation

- 1st Capturing Group `(findTestObject\(((?=.*+').*\))`
 - `findTestObject` matches the characters `findTestObject` literally (case sensitive)
 - `\(` matches the character `(` with numeric value `4010` (`508` or `2816`) literally
 - 2nd Capturing Group `((?=.*+').*\))`
 - `'` matches the character `'` with numeric value `3910` (`478` or `2716`) literally
 - `.*+` matches any character (except for line terminators)
 - `+` matches the previous token one or more times, as many times as possible, giving characters back as needed (greedy)
 - `'` matches the character `'` with numeric value `3910` (`478` or `2716`) literally

Match Information

Match 1	62-109	findTestObject('All_Users/view_all_users_btn'))
Group 1	62-109	findTestObject('All_Users/view_all_users_btn'))
Group 2	77-107	'All_Users/view_all_users_btn'
Match 2	125-173	findTestObject('All_Users/search_input'),'david)
Group 1	125-173	findTestObject('All_Users/search_input'),'david)
Group 2	140-164	'All_Users/search_input'

Abbildung 4: Das `fto_param_str_pattern`-Muster extrahiert die `findTestObject()`-Methode mit dem String-Argument, das für die Transformation benötigt wird.

Die folgende Tabelle zeigt die wichtigsten Regex Muster, die im Transpilationsprozess zum Einsatz kommen:

#	Pattern Name	Regex	Zweck
1	comment_pattern	/*[^*]**(?:[/*][^*]**+)?/	Block-Kommentare entfernen
2	private_method_pat	private void.*{\\n[\\s\\w\\.\\(\\)=\\\"\\', '\\/\\[+@\\-\\]\\\\\\;\\<{\\}*\\n}	Private Methoden extrahieren
3	katalon_lines_pattern	\\/* \\/\\/\\.+ \\/*\\.+ WebUI\\.+\\n\\.+ WebUI\\.+ CustomKeywords.*	Relevante Zeilen filtern
4	katalon_code_pattern	(\\w+)\\. (\\w+)\\(((.*)\\)	Teilt Code in Klasse, Methode, Parameter
5	fto_as_param_pat	(findTestObject\\(\\.*\\))(?=,)	Erkennt findTestObject() als <i>erster</i> Parameter mit Lookahead auf folgendes Komma
6	ftd_as_param_pat	(findTestData\\(\\.*\\))(?=,)	Erkennt findTestData() als <i>erster</i> Parameter mit Lookahead auf folgendes Komma
7	param_pattern	,\\s+(?=false) (?!\\) ,\\s(?!Fail\\.*) (?![a-zA-Z]),\\s(?!\\s)(?![a-zA-Z]) ,\\s(?!null) ,\\s+(?!=\\[)	Parameter teilen, wenn mehrere vorhanden
8	fto_param_str_pattern	(findTestObject\\(\\('.*'\\).*\\))	Extrahiert findTestObject() mit String-Argument zur Transformation
9	ftd_param_str_pattern	findTestData\\(\\('.*'\\).*\\)\\.getValue\\(\\(\\.+\\)\\)	Extrahiert findTestData() mit String und getValue()-Argument
10	GlobalVariable pattern	GlobalVariable\\.([A-Za-z_][A-Za-z0-9_]*)	Global Variablen normalisieren
11	abn_test_pat	String\\s\\w+\\s= if\\(TestObject\\s\\w+\\s=	Custom Code erkennen

Tabelle 1: Übersicht der Regular Expression (Regex) Muster im Transpilationsprozess

5 Implementierung

Dieses Kapitel beschreibt die technische Umsetzung des Migrationswerkzeugs: seine Ordnerstruktur, die Trennung zwischen Migrations- und Laufzeitcode sowie die Aufgaben der einzelnen Pipeline-Module.

5.1 Projektstruktur des Migrators

Das Werkzeug ist in folgende logische Ordnerstruktur unterteilt:

```
src/
├─ pipeline/                                # Build-Time: Migrations-Pipeline
│   ├─ test_script_scanner.py              # Orchestrierung: Scannt Katalon Skripte
│   ├─ test_transpiler.py                  # Groovy zu Python Transpilation
│   ├─ test_assembler.py                   # Pytest Code-Generierung
│   ├─ object_repo_converter.py            # Object Repository XML zu JSON Konvertierung
│   ├─ global_vars_generator.py            # Global Variables Profile Generator
│   ├─ variables_extractor.py              # Test Case Variables zu Python
│   ├─ copy_runtime_files.py               # Runtime-Dateien kopieren + Config erzeugen
│   └─ __init__.py
├─ runtime/                                # Runtime: Wird ins Ziel-Projekt kopiert
│   ├─ base_test.py                        # Selenium WebDriver Base Class
│   ├─ katalon_helpers.py                  # Katalon-kompatible WebDriver-Helfer
│   └─ __init__.py
└─ utils/                                  # Build-Time Utilities
    ├─ file_utils.py                       # Datei-Operationen + Variable-Extraktion
    ├─ string_utils.py                     # Identifier-Normalisierung
    ├─ xml_utils.py                        # Generische XML zu JSON Konvertierung
    └─ __init__.py
```

Listing 3: Ordnerstruktur des Migrationstools mit Trennung zwischen Build-Time und Runtime.

Die Struktur trennt die Verantwortungen: `pipeline/` führt die gesamte Transformation während der Build-Time durch, `runtime/` bündelt die ins Ziel-Projekt zu kopierenden Abhängigkeiten, und `utils/` stellt gemeinsame Hilfsfunktionen bereit.

5.1.1 Build-Time vs. Runtime

Damit der Migrator funktioniert muss zwischen Build-Time und Runtime Code unterschieden werden. Build-Time bezeichnet die Ausführung des Migrators selbst. Runtime bezeichnet dagegen den späteren Zeitpunkt, zu dem die generierten Tests im Ziel-Projekt mit Selenium ausgeführt werden.

Die Dateien im `src/runtime/`-Ordner sind keine Bestandteile der Migrationspipeline, sondern werden erst im Ziel-Projekt benötigt. Sie enthalten Selenium-WebDriver-Abhängigkeiten, die zum Zeitpunkt der Migration nicht installiert sind. Würden sie als gewöhnliche Python-Dateien im Repository liegen, würde der Compiler beim Analysieren des Migrator-Projekts Importfehler melden.

Deswegen tragen die Runtime-Dateien die Endung `.template` und werden vom Compiler ignoriert. `copy_runtime_files.py` kopiert sie während der Build-Time in das Ziel-Projekt und entfernt dabei die Endung automatisch.

5.2 Pipeline-Module im Detail

Die Migrationspipeline besteht aus sechs spezialisierten Modulen, die nacheinander von `main.py` aufgerufen werden. Jedes Modul ist für genau eine Transformation verantwortlich.

`test_suite_translator.py` ist der Einstiegspunkt der Pipeline. Es durchläuft rekursiv den `Scripts/-`Ordner des Katalon-Projekts, filtert `.groovy`-Dateien heraus und delegiert jede Datei an `test_transpiler.py` und `test_assembler.py`. Dabei wird die Ordnerstruktur des Katalon-Projekts in der Zielstruktur `src/tests/` gespiegelt. Tests, die nicht vollständig übersetzbar sind, werden in `src/unreadable_tests/` abgelegt.

`test_transpiler.py` führt die eigentliche syntaktische Transformation durch. Es wendet die in Tabelle 1 dokumentierten Regex Muster auf den Groovy-Code an, erkennt Katalon-Methodeaufrufe und übersetzt sie schrittweise in Python-Äquivalente. Das Ergebnis ist eine Liste von bereits transformierten Code-Zeilen.

`test_assembler.py` nimmt die transpilierten Zeilen und baut daraus eine vollständige Python-Pytest-Testklasse zusammen. Es fügt die notwendigen Imports hinzu (`pytest`, `selenium`, `katalon_helpers`, `base_test`, `global_variables`), erzeugt die Klassenstruktur mit `BaseTest`-Vererbung und verpackt die Testlogik in eine `test_-`Methode.

`object_repo_converter.py` konvertiert alle `.rs`-Dateien im `Object Repository/-`Ordner von XML nach JSON. Die interne `WebElementEntity`-Struktur bleibt erhalten; lediglich das Dateiformat ändert sich. Das Ergebnis wird unter `src/object_repository/` abgelegt.

`global_vars_generator.py` liest `Profiles/default.glbl` (eine XML-Datei mit `GlobalVariableEntity`-Einträgen) und erzeugt daraus `src/profiles/global_variables.py` — eine Python-Klasse `GlobalVariables` mit Klassenvariablen für jede globale Variable des Katalon-Projekts.

`variables_extractor.py` liest die `.tc`-Metadateien der Test Cases und extrahiert testeigene Variablen. Für jeden Test mit Variablen wird eine eigene Python-Datei unter `src/variables/` erzeugt, die die Variablen als Python-Klasse bereitstellt.

`copy_runtime_files.py` schließt die Migration ab: Es kopiert `base_test.py.template` und `katalon_helpers.py.template` in `src/runtime/` des Ziel-Projekts (mit Endungs-Entfernung) und generiert die Konfigurationsdateien `pytest.ini`, `requirements.txt`, `.gitignore` und `README.md` im Wurzelverzeichnis.

6 Evaluation

Zur Evaluation des Migrationswerkzeugs wurde das Katalon-Beispielprojekt `sample-website-katalon-tests` verwendet. Es enthält reale End-to-End-Tests, ein `Object Repository`, ein

globales Variablen-Profil und eine eingebundene CSV-Datendatei. Die Migration wurde auf einem Windows-11-Rechner mit Python 3.13.0 und pytest 8.4.1 durchgeführt.

6.1 Migrationsergebnis

Die Ausführung des Migrators (`python main.py`) liefert folgendes Ergebnis:

Komponente	Anzahl	Beschreibung
Übersetzte Testskripte	5	Alle .groovy-Dateien aus Scripts/ erfolgreich übersetzt
Nicht übersetzbare Tests	0	Kein Test in src/unreadable_tests/ abgelegt
Object Repository Dateien	9	Alle .rs XML-Dateien zu JSON konvertiert
Variablen-Dateien	1	Testeigene Variablen aus .tc-Metadateien extrahiert
Runtime-Dateien	2	base_test.py und katalon_helpers.py ins Ziel kopiert
Konfigurations-Dateien	4	pytest.ini, requirements.txt, .gitignore, README.md
Datendateien	1	users.csv direkt ins Ziel-Projekt kopiert

Tabelle 2: Migrationsergebnis: Alle Komponenten des Katalon-Projekts wurden vollständig überführt.

Alle 5 Testskripte wurden ohne Fehler übersetzt. Kein Test wurde in `unreadable_tests/` abgelegt, was einer Übersetzungsquote von 100% entspricht.

6.2 Strukturelle Integrität

Nach der Migration wurde das Ziel-Projekt mit `pytest --collect-only` geprüft, ohne einen Browser oder die Zielanwendung zu starten. Pytest konnte alle generierten Dateien fehlerfrei importieren und alle 5 Tests entdecken:

```
collected 5 items
```

```
src/tests/Menus/test_navigation_bar.py::Test_navigation_bar::test_navigation_bar
src/tests/Users/
test_filter_for_admins.py::Test_filter_for_admins::test_filter_for_admins
```

```
src/tests/Users/
test_search_for_david_kim.py::Test_search_for_david_kim::test_search_for_david_kim
src/tests/Users/
test_show_active_viewers.py::Test_show_active_viewers::test_show_active_viewers
src/tests/Util/test_login.py::Test_login::test_login
```

```
===== 5 tests collected in 0.27s =====
```

Keine Import-Fehler, keine unaufgelösten Abhängigkeiten. Die generierten Module sind syntaktisch gültiges Python, alle Imports (katalon_helpers, base_test, GlobalVariables) sind auflösbar.

6.3 Codequalität: Vorher-Nachher-Vergleich

Am Beispiel des Tests `filter_for_admins` lässt sich der Unterschied zwischen Original und generiertem Code zeigen:

```
// Katalon (Groovy) – 37 Zeilen, davon 18 Boilerplate-Imports
import static com.kms.katalon.core.testdata.TestDataFactory.findTestData
import static com.kms.katalon.core.testobject.ObjectRepository.findTestObject
// ... 16 weitere Import-Zeilen ...

WebUI.callTestCase(findTestCase('Util/login'), [:], FailureHandling.STOP_ON_FAILURE)
WebUI.verifyTextPresent('View All Users', false)
WebUI.click(findTestObject('All_Users/view_all_users_btn'))
WebUI.selectOptionByValue(findTestObject('All_Users/roles_dropdown'), 'Admin', false)
WebUI.verifyTextPresent(findTestData('users').getValue(3, 1), false)

# Generierter Python-Code – 22 Zeilen, 7 Imports (konsolidiert)
import pytest
import src.runtime.katalon_helpers as kh
from src.runtime.base_test import *
from src.profiles.global_variables import GlobalVariables as GlobalVariable
from selenium.webdriver.support.ui import Select

class Test_filter_for_admins(BaseTest):
    def test_filter_for_admins(self):
        assert 'View All Users' in self.driver.page_source
        kh.find_katalon_test_object(self.driver, 'All_Users/
view_all_users_btn').click()
        Select(kh.find_katalon_test_object(self.driver, 'All_Users/
roles_dropdown')).select_by_value('Admin')
        assert kh.find_katalon_test_data(self.driver, 'users', 3, 1) in
self.driver.page_source
```

Der Boilerplate-Anteil sinkt von 18 auf 7 Importzeilen. Die Testlogik ist in standardmäßigem Python mit pytest-Konventionen formuliert und benötigt keine proprietären Katalon-Keywords mehr.

6.4 Aufwandsvergleich

Die automatisierte Migration des Beispielprojekts dauerte unter einer Sekunde. Zum Vergleich: Eine manuelle Migration hätte folgende Einzelschritte erfordert:

- Verstehen der Katalon-Verschachtelungen (Scripts ↔ Test Cases) und Zuordnen der Dateien
- Übersetzen von 5 Groovy-Testskripten nach Python (ca. 30–60 Minuten pro Test)
- Manuelles Konvertieren von 9 XML-Objektdateien in lesbare Selenium-Locator-Definitionen
- Extrahieren der GlobalVariables aus der .gbl-Datei und Erstellen einer Python-Klasse
- Aufsetzen der Projektstruktur (pytest.ini, requirements.txt, Ordner, init.py-Dateien)

Der geschätzte manuelle Aufwand liegt bei 8-12 Stunden für das Beispielprojekt. Mit wachsender Testanzahl skaliert der Migrator linear, während der manuelle Aufwand überproportional steigt, da Querbezüge (Variablen, Objekte, Daten) zunehmend komplex werden.

7 Diskussion

8 Fazit & Ausblick

Quellenverzeichnis

- [1] Katalon, Inc., „Katalon | The AI Platform for Software Quality“. Zugegriffen: 21. Juli 2026. [Online]. Verfügbar unter: <https://katalon.com/>
- [2] International Software Testing Qualifications Board, „ISTQB Glossary“. Zugegriffen: 22. Juli 2026. [Online]. Verfügbar unter: <https://glossary.istqb.org/>
- [3] A. Sahay, A. Indamutsa, D. Di Ruscio, und A. Pierantonio, „Supporting the Understanding and Comparison of Low-Code Development Platforms“, in *Proceedings of the 46th Euro-micro Conference on Software Engineering and Advanced Applications (SEAA)*, IEEE, 2020, S. 171–178. doi: 10.1109/SEAA51224.2020.00036.
- [4] C. Shapiro und H. R. Varian, *Information Rules: A Strategic Guide to the Network Economy*. Boston: Harvard Business School Press, 1998.
- [5] pytest development team, „Pytest Documentation: Configuration“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://docs.pytest.org/en/stable/reference/customize.html>
- [6] pytest development team, „Pytest Documentation: Good Integration Practices“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://docs.pytest.org/en/stable/explanation/goodpractices.html>
- [7] Python Packaging Authority, „Python Packaging User Guide: src layout vs flat layout“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://packaging.python.org/en/latest/discussions/src-layout-vs-flat-layout/>
- [8] pytest development team, „Pytest Documentation: Fixtures Reference“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://docs.pytest.org/en/stable/reference/fixtures.html>
- [9] J. E. Friedl, *Mastering Regular Expressions*, 3rd Aufl. Sebastopol, CA: O'Reilly Media, 2006.

Erklärung zur selbstständigen Bearbeitung

Hiermit versichere ich, dass ich die vorliegende Arbeit ohne fremde Hilfe selbständig verfasst und nur die angegebenen Hilfsmittel benutzt habe. Wörtlich oder dem Sinn nach aus anderen Werken entnommene Stellen sind unter Angabe der Quellen kenntlich gemacht.

Ort

Datum

Unterschrift im Original